

PROJECT

PROGRAMMING LANGUAGE PARADIGMS

HIGHER-ORDER FUNCTIONS AND CURRYING IN PYTHON, KOTLIN AND SCALA

Worked by:

- Viola Zeneli
- Sidrit Rrustemi
- Denisa Korra



INTRODUCTION

INTRODUCTION TO HIGHER-ORDER FUNCTIONS AND CURRYING

What Are Higher-Order Functions?

A **higher-order function** is a function that:

- Takes one or more functions as arguments.
- Returns a function as its result.

1. Higher-order functions enable code reuse and reduce redundancy.

2. They allow for the creation of generic functions that can be used with different functions as arguments.

3. An example of a higher-order function is `map`, which applies a function to each element of a list.

What is Currying?

Currying is the technique of converting a function that takes multiple arguments into a sequence of functions, each taking a single argument.

For example: If a function add(a, b) takes two arguments, we transform it into add(a)(b) where a is provided first, and b is provided later.

1. Currying transforms a function with multiple arguments into a sequence of functions, each taking a single argument.

2. It allows for partial application of arguments, creating specialized functions for specific use cases.

3. Currying enhances code readability and simplifies complex behavior.

WHY HIGHER-ORDER FUNCTIONS AND CURRYING MATTER IN FUNCTIONAL PROGRAMMING AND MODERN LANGUAGES

Functional Programming

- **Reusability**: Higher-order functions allow us to abstract behaviors, making code more reusable.
- **Modularity**: Functions can be composed easily, leading to clean, modular code.
- **Flexibility**: Currying allows partial application of functions, enabling flexibility in how arguments are provided.

Encapsulation of Behavior:

Allows customizable behavior in a clean, reusable manner.

Example: map, filter, and reduce functions in functional programming rely on higher-order functions.

Modern Languages

Partial Application:

- Currying helps in creating partially applied functions, which makes it easier to specialize functions for specific use cases.
- These techniques facilitate **immutability,** declarative style, and easier composition of complex behavior.

Enhanced Readability:

- Currying allows you to structure functions for more **clarity** and **flexibility**

HOF & CURRYING IN PYTHON

DENISA KORRA

PARADIGMS PROJECT

HOFS IN PYTHON

Functions in Python are **first-class citizens**, meaning they can be:

- Assigned to variables.
- Passed as arguments to other functions.
- Returned from other functions.

```python
def apply_function(func, value):  1 usage
    return func(value)

def square(x):  1 usage
    return x * x

result = apply_function(square, value: 4)

print(result)

```

```
16

Process finished with exit code 0
```

CURRYING IN PYTHON

- Python does NOT support currying natively but we can simulate it using closures.

Closure in Python allows a function to remember the variables from its enclosing scope even after the outer function has finished executing.

Func.py Currying.py

```
1  def curried_add(a):  1 usage
2      def add_b(b):
3          return a + b
4      return add_b
5
6  add_five = curried_add(5)
7  print(add_five(3))
8
```

```
8

Process finished with exit code 0
```

DIFFERENCES BETWEEN HOFS AND CURRYING

Concept	Higher-Order Functions (HOFs)	Currying
How It Works	You can pass functions as arguments or return them from other functions.	You create a sequence of functions, each accepting one argument and returning another function.
Purpose	To abstract and manipulate behavior by passing functions around.	To partially apply arguments to a function step-by-step.
Use Case	Used for function composition, abstraction, and higher-level operations like map(), filter(), reduce().	Used when you want to pre-set one or more arguments of a function and apply the remaining arguments later.
Real-World Example	- Event handlers in GUI programming where behavior is passed as a function. - Functional libraries like map(), filter(), reduce() which take functions as arguments.	- Logging where a part of the function (like a log level) is set, and other values (like message) are passed later. - Mathematical operations like add(a)(b) where a is fixed first, and b is added later.
When to Use	When you need flexibility, reuse code, or compose multiple operations into a single function.	When you want to reuse a function with some arguments pre-set, making the function more specialized for future use.

WHEN AND WHY EACH IS USED IN REAL-WORLD CODE

HOFs in Real-World Code

Map, Filter, and Reduce:

- These functions are **built-in higher-order functions** in Python. They take functions as arguments and return the result of applying that function to an iterable (list, tuple, etc.).
- They allow **concise** and **functional** programming, where logic can be abstracted and reused.
- You can pass custom functions into **map()**, **filter()**, or **reduce()** to apply specific behaviors on data collections.

Map()

The **map()** function applies a given function to **each item** in an iterable and returns a **new iterable** (list, tuple) of the results.

```
1   nums = [1, 2, 3, 4]
2   doubled = list(map(lambda x: x * 2, nums))
3
4   print(doubled)
5
```

```
[2, 4, 6, 8]

Process finished with exit code 0
```

HOFS IN REAL-WORLD CODE

Filter()

The **filter()** function filters out elements from an iterable based on a condition defined by a function. It **returns** a **new iterable** (filtered list) containing only the elements that **satisfy** the given condition.

```
nums = [1, 2, 3, 4, 5, 6]
even_nums = list(filter(lambda x: x % 2 == 0, nums))

print(even_nums)
```

```
[2, 4, 6]

Process finished with exit code 0
```

Reduce()

The **reduce()** function **reduces** an iterable (list, tuple) to a **single value** by applying a function that takes two arguments, combining them into one. It **keeps applying** the function cumulatively to the elements of the iterable.

```
from functools import reduce

nums = [1, 2, 3, 4]
sum_of_nums = reduce(lambda x, y: x + y, nums)

print(sum_of_nums)
```

```
10

Process finished with exit code 0
```

WHEN AND WHY EACH IS USED IN REAL-WORLD CODE

Currying in Real-World Code

Logging:

- A logging function might use currying to set the log level once and then accept log messages as needed later.

Why Currying:

- It allows for **partial application**, where you set up part of the function (like the log level) once, and you can reuse that function for logging different messages.

Logging

```python
# Curried logger function
def log_level(level):  1 usage
    def log_message(message):
        print(f"[{level}] {message}")
    return log_message

info_logger = log_level("INFO")
info_logger("This is an info message.")
```

```
[INFO] This is an info message.

Process finished with exit code 0
```

CUSTOM HIGHER-ORDER FUNCTION

Function that takes another function as an argument

```
def function(func, value):  1 usage
    return func(value)

def double(x):  1 usage
    return x * 2

print(function(double,  value: 3))
```

```
6

Process finished with exit code 0
```

Function that returns another function

```
def multiplier(factor):  1 usage
    def multiply_by(x):
        return x * factor
    return multiply_by

multiply_by_2 = multiplier(2)
print(multiply_by_2(5))
```

```
10

Process finished with exit code 0
```

PARTIAL APPLICATION EXAMPLE

```python
from functools import partial

def power(base, exponent):  1 usage
    return base ** exponent

square = partial(power, exponent=2)
print(square(5))
```

```
25

Process finished with exit code 0
```

In Python, partial application can be achieved using the functools.partial() function, which is a built-in utility for fixing a certain number of arguments of a function, creating a new function that takes fewer arguments.

How It Works:
Partial application means applying a function to a subset of its arguments.
Instead of repeatedly passing the same argument every time you call a function, you can "fix" that argument and generate a new function that only requires the remaining arguments.

Higher-Order Function (HOF) with Function Composition

```python
# Higher-Order Function Example with Function Composition
def compose(func1, func2):  1 usage
    return lambda x: func2(func1(x))  # Apply func1 first, then func2

# Two simple functions to manipulate strings
def to_uppercase(text):  1 usage
    return text.upper()

def add_exclamation(text):  1 usage
    return text + "!"

# Using the HOF to compose functions
shout = compose(to_uppercase, add_exclamation)
result = shout("hello")
print(result)
```

```
HELLO!

Process finished with exit code 0
```

Currying Example with Custom Math Operation

```python
# Currying Example for Mathematical Operations
def curried_math(op):  3 usages
    def operate(a):
        def apply_operation(b):
            if op == 'add':
                return a + b
            elif op == 'subtract':
                return a - b
            elif op == 'multiply':
                return a * b
            else:
                return None
        return operate

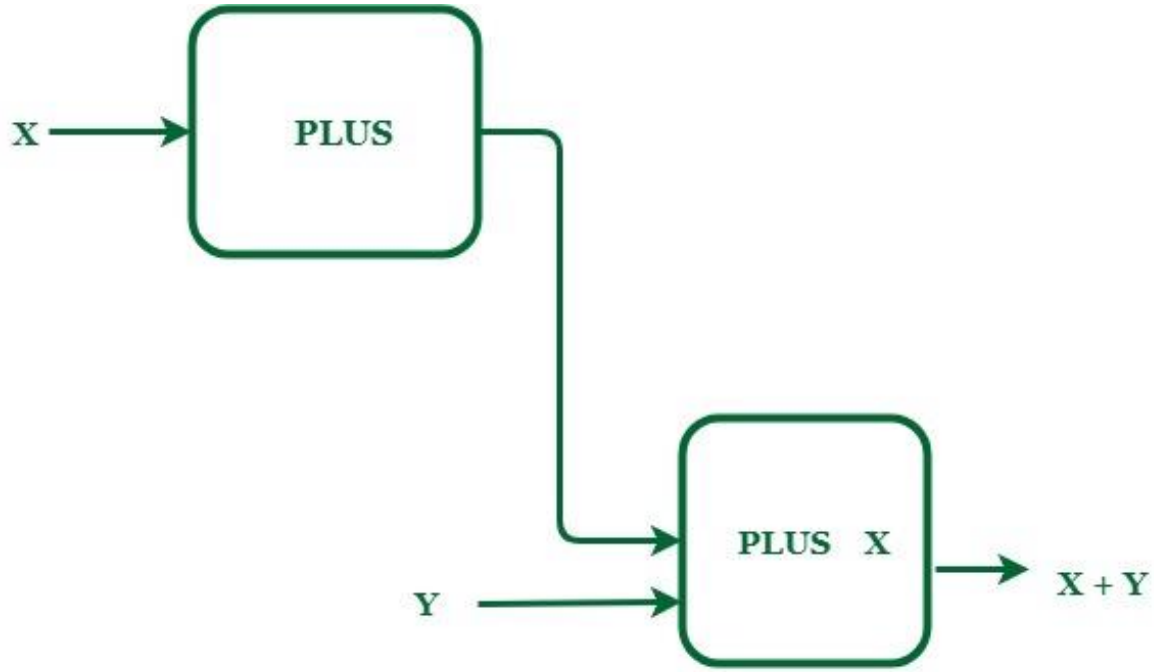
# Create curried functions for specific operations
add_five = curried_math('add')(5)
subtract_three = curried_math('subtract')(3)
multiply_by_two = curried_math('multiply')(2)

# Apply the operations
print(add_five(3))  # Output: 8
print(subtract_three(10))  # Output: 7
print(multiply_by_two(4))  # Output: 8
```

Let's take an example, PLUS is a function which adds two numberWe wish to add two numbers X and Y. X will be the input to PLUS function which returns a function named PLUS X.
PLUS X function takes one number and add X to it. Now input to this function will be Y. Final output will be X + Y.



Normal Implementon of Function



Implemention of Curried Function

ADVANTAGES AND DISADVANTAGES OF HIGHER ORDERED FUNCTIONS IN PYTHON

Advantages	Disadvantages
1. Code Reusability: HOFs allow you to create **generic functions** that can be reused with different behavior by passing different functions.	**1. Complexity**: HOFs can introduce **complexity** and make code harder to understand, especially for beginners. It can be difficult to track the flow of execution when functions are passed around.
2. Modularity: HOFs allow for **modular code** by abstracting behavior into functions. You can pass different functions to achieve different behaviors without changing the structure.	**2. Performance Overhead**: Using HOFs, especially with functions like **map()**, **filter()**, or **reduce()**, can add **extra overhead** when compared to using simple loops, especially in tight loops or performance-sensitive applications.
3. Function Composition: HOFs facilitate **function composition**, where you can combine small functions to create complex operations.	**3. Debugging Difficulty**: Debugging code with HOFs can be more challenging because functions are passed around dynamically, which can make it harder to set breakpoints or track values during execution.
4. Functional Programming Paradigm: They promote **functional programming**, making Python code more flexible, reusable, and easier to maintain.	**4. Can Lead to Nested Functions**: Overuse of HOFs can lead to **deeply nested functions** or callbacks, making the code harder to follow and manage.
5. Cleaner Code: HOFs allow you to **abstract complex logic** and encapsulate it in functions, leading to cleaner, more readable code.	**5. Learning Curve**: Python developers accustomed to **imperative programming** may find it difficult to understand and use HOFs effectively.
6. Higher Flexibility: HOFs provide more **flexibility** and **customization** because they allow you to pass different functions to modify behavior without changing the structure.	**6. Limited Support for Side Effects**: Since functional programming (and HOFs) encourages **pure functions** (no side effects), certain real-world scenarios where side effects are needed (like modifying external states) can be more difficult to handle.
7. Encourages Cleaner Abstractions: HOFs can help **abstract repetitive logic**, reducing code duplication and allowing you to focus on higher-level problem-solving.	**7. Can Obscure Logic**: Overuse of HOFs can make the code harder to reason about, especially when combined with anonymous functions (lambdas). It might obscure the main logic of the program.

ADVANTAGES AND DISADVANTAGES OF CURRYING IN PYTHON

Advantages	Disadvantages
1. Partial Application: Currying allows for **partial application**, where you can **fix** some arguments of a function and apply the rest later. This makes it easier to create specialized functions.	**1. More Complex Syntax**: Currying requires a more **complex syntax** compared to regular functions. In Python, simulating currying can introduce nested functions, which may be harder to follow for beginners.
2. Reusability: Currying allows you to create more **reusable functions** by pre-setting some arguments. You can use the curried function with different remaining arguments, making it highly flexible.	**2. Increased Overhead**: Simulating currying in Python through closures or nested functions can introduce **additional memory overhead** and potentially **slow down performance**, especially if done extensively.
3. More Modular Code: It promotes a more **modular approach** by breaking down a function into smaller, focused functions. Each curried function can represent a part of the operation, improving clarity.	**3. Difficult to Debug**: Debugging curried functions can be challenging, especially when working with closures. Tracking variable values across nested functions can become complicated.
4. Clearer Function Composition: Currying makes it easier to **compose functions** together, leading to cleaner and more expressive code. You can create a pipeline of operations by currying.	**4. Not Native to Python**: Currying is not **natively supported** in Python. You have to simulate it using closures or libraries like functools.partial, making it less intuitive than in other languages like Haskell.
5. Increased Flexibility: You can create **customized** versions of functions by fixing arguments early. This leads to more **flexible** code where specific behavior can be set up upfront.	**5. Can Lead to Overuse**: Overuse of currying can lead to overly **complex function signatures** or too many small functions, making the code more fragmented and harder to manage.
6. Makes Code More Readable in Some Cases: In scenarios like **logging**, **math operations**, or **configuration settings**, currying helps make the code **more readable** by pre-setting common arguments.	**6. Limited to Simple Functions**: Currying is best suited for **simple functions** that can be split into unary functions. For more complex functions, currying may not be the most practical solution.
7. Supports Functional Programming Style: Currying encourages **functional programming** principles in Python, allowing you to use concepts like **higher-order functions** and **function composition**.	**7. Reduces Performance in Some Cases**: When currying is used with deeply nested functions or closures, it can result in **slower performance** due to the additional function calls and memory consumption.

HOF & CURRYING IN KOTLIN

VIOLA ZENELI

PARADIGMS PROJECT

HIGHER-ORDER FUNCTIONS IN KOTLIN

What are?

In Kotlin, a **higher-order function** is a function that either:

- Takes another function as a parameter, or
- Returns a function as a result.

This is made possible because **functions are first-class citizens** in Kotlin — meaning they can be:

- Stored in variables
- Passed to other functions
- Returned from functions
- Stored in data structures

Why are important?

- Kotlin is a multi-paradigm language, but embraces functional programming. HOFs make code more concise and composable.
- Using HOFs allows reusing logic with different behaviors and reducing code repetition.
- You can pass in your own function or lambda to dynamically control behavior (e.g., sorting, filtering, reducing).

KOTLIN SYNTAX SUPPORT

EXAMPLE 1: FUNCTION AS PARAMETER

```
// Here, op is a function type (Int, Int) -> Int
fun calculator(a: Int,  b: Int,  op: (Int, Int) -> Int): Int {
    return op(a, b)
}

// its calling
val sum = calculator(3,  4,  { x, y -> x + y })
println(sum)  // Output: 7
```

```
//You can also simplify it using lambda
val sum = calculator(3, 4) { x, y -> x + y }
```

EXAMPLE 2: RETURNING A FUNCTION

```
fun chooseOperation(type: String): (Int, Int) -> Int {
    return if (type == "add") { a, b -> a + b } else { a, b -> a * b }
}

val operation = chooseOperation("add")
println(operation(2, 3))  // Output: 5
```

KOTLIN SYNTAX FEATURES FOR HIGHER-ORDER FUNCTIONS

Lambda Expressions

- Lambda expressions in Kotlin are anonymous functions defined using curly braces. They can take parameters and return values like named functions. Lambdas are essential for defining short, one-off functions and are the building blocks of higher-order functions.

```
val add = { a: Int, b: Int -> a + b }
println(add(2, 3))  // Output: 5

// With type inference
val multiply: (Int, Int) -> Int = { x, y -> x * y }
```

Trailing Lambdas

- When the last parameter of a function is a lambda, Kotlin allows moving the lambda outside of the parentheses. This makes the code more readable and is commonly used in Kotlin's DSLs and collection APIs.

```
// Traditional way
compute(3, 4, { a, b -> a * b })

// Trailing lambda syntax
compute(3, 4) { a, b -> a * b }
```

KOTLIN SYNTAX FEATURES FOR HIGHER-ORDER FUNCTIONS

‘it’ Keyword

- ‘it’ is the implicit name given to the single parameter of a lambda in Kotlin. This makes simple lambda expressions even more concise by avoiding the need to explicitly name the parameter.

```
val numbers = listOf(1, 2, 3, 4)
val evens = numbers.filter { it % 2 == 0 }
println(evens)  // Output: [2, 4]
```

Function References

- Kotlin allows referencing existing functions using the double colon (::) operator. These references can be passed as arguments to higher-order functions, allowing code reuse and avoiding redundant lambdas.

```
fun square(x: Int): Int = x * x
val op: (Int) -> Int = ::square
println(op(4))  // Output: 16

val doubled = listOf(1, 2, 3).map(::square)
```

KOTLIN SYNTAX FEATURES FOR HIGHER-ORDER FUNCTIONS

Lambda with Receiver

- Lambdas with receiver allow you to call methods on a receiver object inside the lambda without using a qualifier. These are used heavily in Kotlin DSLs.

```
val greet: String.() -> String = { "Hello, $this!" }
println(”Viola".greet())  // Output: Hello, Viola!
```

Returning Functions

- Kotlin supports returning lambdas from functions, enabling closures. Returned functions can capture and use variables from their defining scope.

```
fun makeAdder(x: Int): (Int) -> Int = { y -> x + y }
val addFive = makeAdder(5)
println(addFive(3))  // Output: 8
```

BUILT-IN HIGHER-ORDER FUNCTIONS IN KOTLIN

- Kotlin's built-in higher-order functions simplify working with collections through functional programming patterns. Below is a detailed explanation of each key function including what it does, when it's useful, and example usage.

- **map()**

Applies a transformation function to each element of a collection and returns a new list with the results. Useful for converting one type of data to another or modifying each item.

Example:

```
val numbers = listOf(1, 2, 3)
val doubled = numbers.map { it * 2 }
println(doubled)  // Output: [2, 4, 6]
```

- **filter()**

Selects elements that match a given condition (predicate). Returns a list of elements that satisfy the condition.

Example:

```
val numbers = listOf(1, 2, 3, 4)
val evens = numbers.filter { it % 2 == 0 }
println(evens)  // Output: [2, 4]
```

- **forEach()**

Applies a function to each element in the collection. It does not return a new list; instead, it's used for side effects like printing or logging.

Example:

```
val names = listOf("Ana", "Ben", "Cleo")
names.forEach { println(it) }
```

BUILT-IN HIGHER-ORDER FUNCTIONS IN KOTLIN

- **fold()**

Reduces a collection to a single value by accumulating the result from left to right, starting with an initial value. Useful for summing, concatenating, or aggregating elements.

Example:

```
val nums = listOf(1, 2, 3, 4)
val total = nums.fold(0) {acc, n -> acc + n}
println(total)  // Output: 10
```

- **reduce()**

Similar to fold, but does not take an initial value — it uses the first element of the list. It returns a single result from the reduction.

Example:

```
val nums = listOf(1, 2, 3, 4)
val product=nums.reduce {acc,n -> acc* n}
println(product)  // Output: 24
```

- **any()**

Returns true if at least one element in the collection satisfies the given condition.

Example:

```
val list = listOf(1, 2, 3)
println(list.any { it > 2 })  // Output: true
```

BUILT-IN HIGHER-ORDER FUNCTIONS IN KOTLIN

- **all()**

Returns true if all elements satisfy the condition.

Example:

```
val list = listOf(1, 2, 3)
println(list.all { it > 0 })  // Output: true
```

- **take(n)**

Returns a new list containing the first n elements of the collection.

Example:

```
val list = listOf(1, 2, 3, 4, 5)
println(list.take(3))  // Output: [1, 2, 3]
```

- **none()**

Returns true if no elements satisfy the condition.

Example:

```
val list = listOf(1, 2, 3)
println(list.none { it < 0 })  // Output: true
```

- **drop(n)**

Returns a new list with the first n elements removed.

Example:

```
val list = listOf(1, 2, 3, 4, 5)
println(list.drop(3))   // Output: [4, 5]
```

CURRYING IN KOTLIN

Definition of Currying

- Currying is the process of transforming a function that takes multiple arguments into a sequence of functions, each taking a single argument. Instead of calling a function with all parameters at once, you call a series of functions that return other functions.

  For example, a function of type (A, B) -> C becomes: A -> (B -> C).

  This technique enables function reuse and helps in building more modular and readable code.

How Currying Is Achieved In Kotlin?

- Kotlin does not have native support for automatic currying like Haskell or Scala, but currying can be implemented manually using higher-order functions. You define a function that returns another function, each taking one argument.

  Example:

```
fun multiply(x: Int): (Int) -> Int = { y -> x * y }
val double = multiply(2)
println(double(5))  // Output: 10
```

  This simulates a curried function by returning a lambda.

CURRYING VS PARTIAL APPLICATION

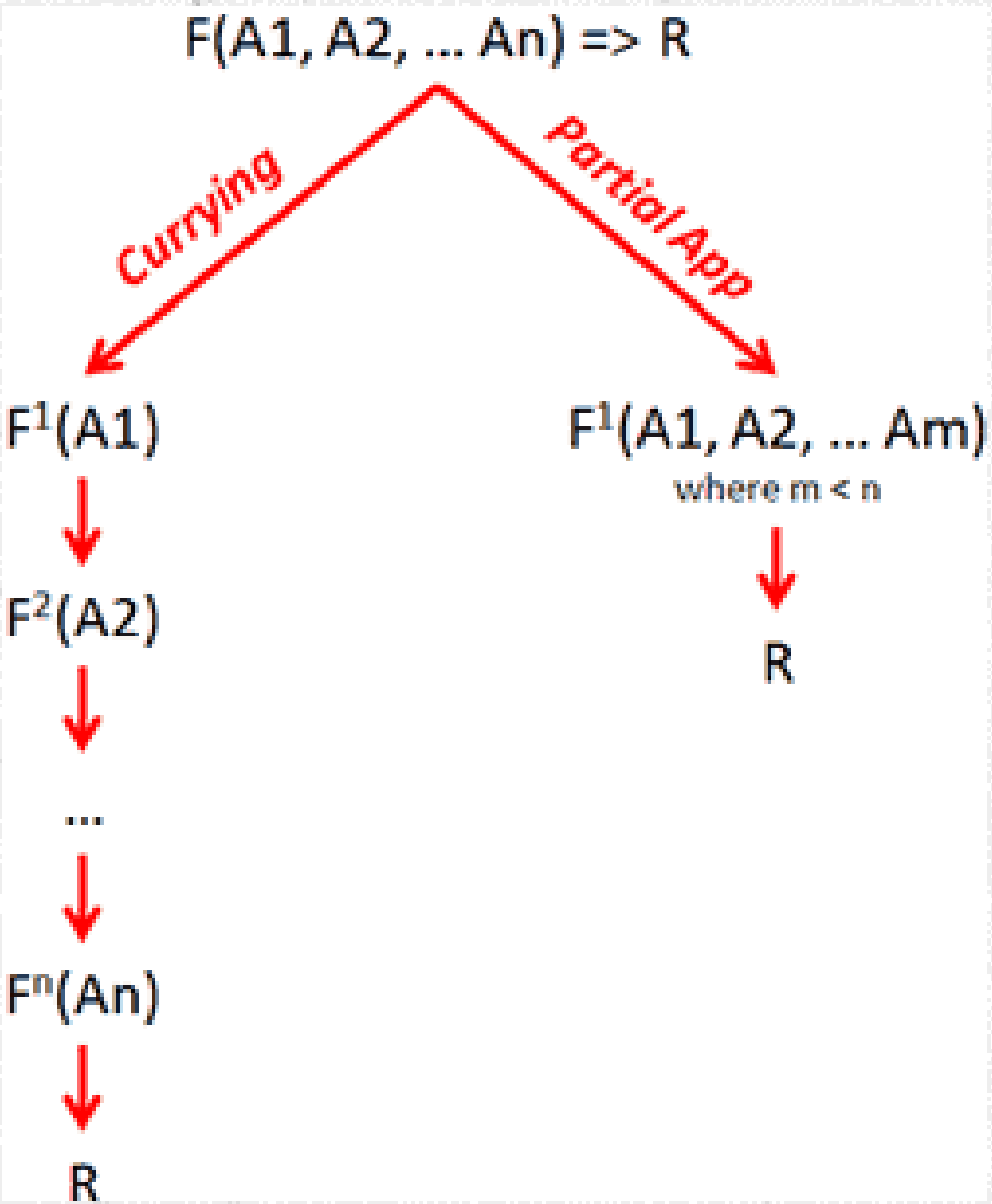
- Currying transforms a multi-parameter function into a chain of single-parameter functions.
- Partial application fixes a few arguments of a function and returns a new function with fewer parameters.

Difference:

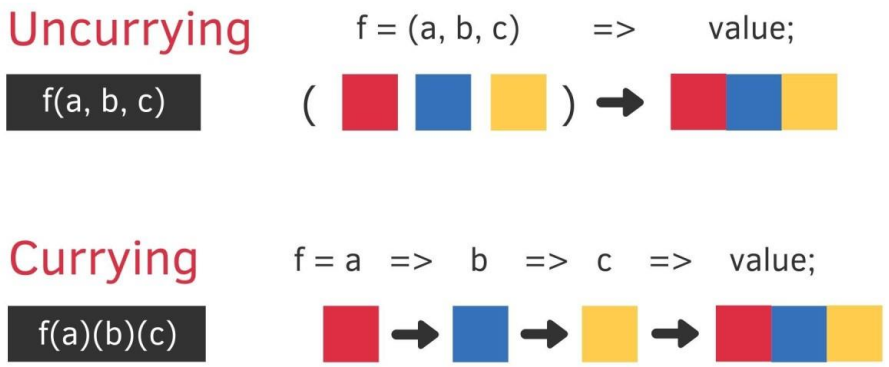
- Currying always transforms to a series of unary functions.
- Partial application can stop at any stage with any number of arguments fixed.

Example of partial application in Kotlin:

```
fun add(a: Int, b: Int) = a + b
val addFive = { x: Int -> add(5, x) }
println(addFive(3))  // Output: 8
```



MANUAL CURRYING IN KOTLIN



Manual currying in Kotlin involves writing nested functions or lambdas that take one parameter at a time.

Example:

```
fun curriedSum(a: Int): (Int) -> (Int) -> Int = { b -> { c -> a + b + c } }
val result = curriedSum(1)(2)(3)
println(result)  // Output: 6
```

Each invocation returns a new function expecting the next argument.

USE CASES OF HOF AND CURRYING IN KOTLIN

Use Case	HOFs	Currying
DSLs	Used to build readable config-like code (e.g., Jetpack Compose).	Rarely used.
Callbacks	Used a lot for passing logic in UIs or APIs.	Not used here.
Events	Common for click listeners, etc.	Not practical.
Data Pipelines	Chain functions like map, filter, etc.	Create flexible pipelines with partial values.
Custom Functions	Pass lambdas to reuse logic.	Return functions with pre-filled values.
Delayed Execution	Run lambdas later (e.g., with inline).	Wait until all arguments are supplied.

HOF & CURRYING IN SCALA

SIDRIT RRUSTEMI

PARADIGMS PROJECT

HIGHER-ORDER FUNCTIONS IN SCALA

What are HOFs?

In Scala, a **higher-order function** is a function that either:

- Takes another function as a parameter, or
- Returns a function as a result.

This is made possible because **functions are first-class citizens** in Scala — meaning they can be:

- Stored in variables
- Passed to other functions
- Returned from functions

Why are they important?

- Scala is a multi-paradigm language, but embraces functional programming. HOFs make code more concise and composable.
- Using HOFs allows reusing logic with different behaviors and reducing code repetition.
- You can pass in your own function or lambda to dynamically control behavior (e.g., sorting, filtering, reducing).

FUNCTION AS PARAMETER

EXAMPLE 1

```
def applyOperation(x: Int, operation: Int => Int): Int = {
  operation(x)
}

val square = (n: Int) => n * n
val result = applyOperation(5, square)

println(result)  // Output: 25
```

```
// You can also pass the lambda directly
val result2 = applyOperation(6, x => x + 10)
println(result2)  // Output: 16
```

EXAMPLE 2

```
def calculate(a: Int, b: Int, op: (Int, Int) => Int): Int = {
  op(a, b)
}

val sum = calculate(3, 4, (x, y) => x + y)
val product = calculate(3, 4, _ * _)

println(s"Sum: $sum")  // Output: Sum: 7
println(s"Product: $product")  // Output: Product: 12
```

RETURNING A FUNCTION

EXAMPLE 1

```
def multiplier(factor: Int): Int => Int = {
  (x: Int) => x * factor
}

val triple = multiplier(3)  // triple is a function: Int => Int
println(triple(4))  // Output: 12
```

EXAMPLE 2

```
def greetWith(greeting: String): String => String = {
  (name: String) => s"$greeting, $name!"
}

val sayHello = greetWith("Hello")
val sayWelcome = greetWith("Welcome")

println(sayHello("Scala"))  // Output: Hello, Scala!
println(sayWelcome("Sidrit"))  // Output: Welcome, Sidrit!
```

TRAILING LAMBDAS IN SCALA

Trailing Lambdas

- When the last parameter of a function is a lambda, Scala allows moving the lambda outside of the parentheses.

```
object LambdaExample extends App {
  def compute(a: Int, b: Int, op: (Int, Int) => Int): Int =
op(a, b)

  // Traditional way
  val result1 = compute(3, 4, (a: Int, b: Int) => a * b)
  println(result1)

  // Trailing lambda syntax
  val result2 = compute(3, 4) { (a: Int, b: Int) => a * b }
  println(result2)
}
```

“_” AS A PLACEHOLDER IN SCALA

“_” Placeholder

- When you want to write a simple anonymous function but don't want to explicitly name the parameters, you can use _ as a placeholder.
- Scala will replace each _ with a different parameter in order.

```
def compute(a: Int, b: Int, op: (Int, Int) => Int): Int
= op(a, b)

// Using named params
val sum1 = compute(3, 4, (x, y) => x + y)

// Using _ placeholder syntax
val sum2 = compute(3, 4, _ + _)

println(sum1)  // Output: 7
println(sum2)  // Output: 7
```

BUILT-IN HIGHER-ORDER FUNCTIONS IN SCALA

1. map

- Applies a function to each element, returns a new collection of results.

```
val nums = List(1, 2, 3, 4)


// Using explicit parameter x
val doubled = nums.map(x => x * 2)


// Using explicit parameter x
val doubled = nums.map(_ * 2)
```

2. filter

- Returns a collection with elements that satisfy a predicate (boolean test).

```
val nums = List(1, 2, 3, 4)
val evens = nums.filter(_ % 2 == 0)  // List(2, 4)
```

BUILT-IN HIGHER-ORDER FUNCTIONS IN SCALA

3. flatmap

- Like map, but the function returns a collection which gets flattened.

```
val words = List("hello", "world")
val chars = words.flatMap(_.toList)
// List('h','e','l','l','o','w','o','r','l','d')
```

4. reduce

- Combines elements using a binary function to produce a single value.

```
val nums = List(1, 2, 3, 4)
val sum = nums.reduce(_ + _)  // 10
```

BUILT-IN HIGHER-ORDER FUNCTIONS IN SCALA

5. fold

- Similar to reduce but allows specifying a starting (initial) value.

```
val nums = List(1, 2, 3)
val sum = nums.fold(0)( _ + _ )  // 6
```

6. foreach

- Applies a function to each element, typically for side effects (returns Unit).

```
val nums = List(1, 2, 3)
nums.foreach(println) // 1 2 3 (in new lines)
```

BUILT-IN HIGHER-ORDER FUNCTIONS IN SCALA

7. exists and forall

- exists checks if any element satisfies a predicate.
- forall checks if all elements satisfy a predicate.

```
val nums = List(1, 2, 3, 4)
nums.exists(_ > 3)  // true
nums.forall(_ > 0)  // true
```

8. sortBy

- Sort elements by a function or natural ordering.

```
val words = List("apple", "banana", "pear")
val sortedWords = words.sortBy(_.length)
 // List("pear", "apple", "banana")
```

CURRYING IN SCALA

Definition of Currying

- Currying is the process of transforming a function that takes multiple arguments into a sequence of functions, each taking a single argument.
- In other words:

```
// Normal function: takes 2 parameters

def add(x: Int, y: Int): Int = x + y

// Curried version: a function that returns another function

def curriedAdd(x: Int)(y: Int): Int = x + y
```

Why use currying?

- **Partial application:** You can fix some arguments and create a new function.
- **Function composition:** It fits well with functional programming patterns.
- **Cleaner syntax:** Especially when passing functions as arguments.

HOW IS CURRYING ACHIEVED IN SCALA?

1. Using Multiple Parameter Lists

- This is the most direct way to define a curried function:

```
def multiply(x: Int)(y: Int): Int = x * y

val timesTwo = multiply(2)_  // Partial application

println(timesTwo(5))  // Output: 10
```

2. Using the curried Method

- You can convert a regular function into a curried one.

```
val add = (x: Int, y: Int) => x + y

val curriedAdd = add.curried

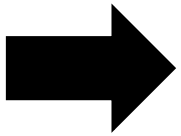

val addFive = curriedAdd(5)

println(addFive(3))  // Output: 8
```

MANUAL CURRYING IN SCALA

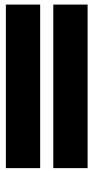
We start with a regular function:

```
val add = (x: Int, y: Int) => x + y
```



Then we manually curry it:

```
def manualCurriedAdd(x: Int): Int => Int = {
  (y: Int) => x + y
}
```



Equivalent to this:

```
def curriedAdd(x: Int)(y: Int): Int = x + y
```

- manualCurriedAdd takes an Int and returns a function Int => Int.
- You call manualCurriedAdd(5) to get a function that adds 5 to whatever you give it.

WHY DO WE USE CURRYING IN SCALA?

1. Partial Application & Impoved Code Reuse

- You can fix some parameters and create a new, more specific function.

```
def multiply(x: Int)(y: Int): Int = x * y


val double = multiply(2) _  // Partially applied

println(double(5))  // Output: 10
```

2. Better Function Composition

- Curried functions work well with higher-order functions and pipelines.

```
val add = (x: Int) => (y: Int) => x + y

val add2 = add(2)

val result = List(1, 2, 3).map(add2)  // List(3, 4, 5)
```

WHERE IS CURRYING MOSTLY USED?

1. Functional Programming

Currying is a core part of functional programming in Scala, Haskell, etc.

2. Frameworks like Play Framework (Scala Web)

In Play Framework, controller actions often use currying to inject dependencies like request handlers or authentication layers:

3. DSLs and APIs

Libraries that build domain-specific languages (DSLs), like Akka HTTP or Cats, often use currying for fluent API design.

4. Partial Functions

Currying makes it easier to create partially applied functions for use in higher-order functions like map, filter, etc.

LANGUAGE COMPARISONS

PARADIGMS PROJECT

COMPARISON OF HOFS IN PYTHON, KOTLIN, AND SCALA

Aspect	Python	Kotlin	Scala
Function as First-Class	Yes, functions can be assigned and passed around.	Yes, functions are first-class citizens.	Yes, core FP concept. Everything is an object including functions.
Lambda Support	Yes, but single-expression lambdas only.	Yes, supports inline multi-parameter lambdas.	Yes, full-featured anonymous functions and blocks.
Built-in HOFs	map, filter, reduce (via functools), sorted, etc.	map, filter, fold, forEach, etc. in standard library.	map, filter, fold, reduce, etc. on collections and Streams.
Function Reference	Use function name without () or use functools.partial.	Use ::functionName syntax.	Pass function names directly, no special syntax needed.
Lambdas with Receivers	Not supported directly.	Yes, supports receivers for DSLs.	Supported using implicit classes or context.

COMPARISON OF CURRYING IN PYTHON, KOTLIN, AND SCALA

Aspect	Python	Kotlin	Scala
Native Support	No native currying. Can be done manually or with functools.partial.	No native support, but can be done manually via nested lambdas.	Yes, supports native currying using multiple parameter lists.
Syntax Simplicity	Manual via nested functions or lambda chains.	Requires explicit nested functions or lambdas.	Simple and elegant syntax: def add(a: Int)(b: Int).
Partial Application	Supported using functools.partial.	Manual only using inline lambdas.	Fully supported as a first-class feature.
Use in Practice	Rare in Python codebases.	Occasional for configuration or closures.	Common in functional-style Scala code.



THANK YOU!

✉ vzeneli23@epoka.edu.al

✉ srrustemi23@epoka.edu.al

✉ dkorra23@epoka.edu.al